Nama : Oktova Yoga Praditia Samosir

Nim : 42324001

Prodi : D3 - Teknologi Informasi

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

kode ini memanggil library yang akan dipakai untuk mengolah data, membuat model jaringan saraf, membagi dataset, dan menampilkan grafik.

- pandas dipakai untuk membaca dan mengelola dataset berbentuk tabel.
- numpy membantu operasi numerik dan array.
- matplotlib.pyplot dipakai untuk membuat grafik.
- tensorflow menjadi framework untuk membangun dan melatih jaringan saraf.
- train_test_split membagi data menjadi data training dan testing.
- standardscaler disiapkan untuk standardisasi fitur, tetapi pada notebook ini belum digunakan.

In [ ]:
```
from google.colab import drive
drive.mount('/content/drive')
```

drive.mount('/content/drive') menghubungkan google colab dengan google drive agar file yang disimpan di drive dapat diakses dari notebook.

In [ ]:
```
!pip install tensorflow
```

menginstall tennsorflow agar dapat digunakan di tab ini

In [ ]:
```
!pip install pandas matplotlib scikit-learn
```

cell ini memasang pandas, matplotlib, dan scikit-learn. ketiganya mendukung tahapan sebelum dan sesudah training: pandas untuk membaca dataset, matplotlib untuk visualisasi, sedangkan scikit-learn menyediakan fungsi pembagian data.

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

library dipanggil kembali agar nama modul dan fungsi tersedia pada sesi eksekusi notebook. pemanggilan ulang tidak membuat model baru dan tidak mengubah dataset.

pada praktik ini, fungsi-fungsi tersebut akan digunakan untuk membaca data advertising, membagi data, dan membangun model mlp menggunakan tensorflow keras.

In [ ]:
```
# Load dataset
dataset = pd.read_csv('/content/advertising.csv')

# Lihat beberapa baris dataset
dataset.head()
```

pd.read_csv() membaca file advertising.csv dan menyimpannya ke variabel dataset. perintah dataset.head() menampilkan lima baris pertama agar isi dan nama kolom dapat diperiksa. dataset memiliki tiga fitur masukan, yaitu tv, radio, dan newspaper, serta satu target sales. tujuan model adalah mempelajari hubungan anggaran iklan pada ketiga media tersebut dengan nilai penjualan. jadi ini termasuk tugas regresi karena output yang diprediksi berupa angka, bukan label kelas. dan lima baris pertama yang ditampilkan adalah head nya.

In [ ]:
```
dataset.describe()
```

dataset.describe() menghasilkan ringkasan statistik untuk setiap kolom numerik. count menunjukkan jumlah data, mean nilai rata-rata, std simpangan baku, min nilai terkecil, kuartil 25%, 50%, 75%, dan max nilai terbesar. pada data untuk setiap kolom, sehingga tidak terlihat kolom numerik yang kekurangan jumlah observasi. rata-rata sales sekitar 15.13, sedangkan rata-rata anggaran tv sekitar 147.04, radio 23.264, dan newspaper 30.554. ringkasan ini membantu memahami skala dan sebaran data sebelum model dilatih.

In [ ]:
```
# Cek nilai null
dataset.isnull().sum()
```

dataset.isnull() memeriksa setiap sel apakah nilainya kosong, lalu .sum() menghitung jumlah nilai kosong pada setiap kolom.

hasil menunjukkan tv, radio, newspaper, dan sales masing-masing memiliki nilai null sebanyak 0. artinya, tidak ditemukan missing value pada kolom-kolom tersebut, sehingga tidak diperlukan penghapusan atau pengisian nilai kosong pada tahap ini.

In [ ]:
```
# Split data menjadi training dan testing set
from sklearn.model_selection import train_test_split

X = dataset.drop(['Sales'], axis=1)
y = dataset['Sales']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

x berisi fitur masukan dengan menghapus kolom sales, sedangkan y berisi target yang akan diprediksi, yaitu sales. fungsi train_test_split() membagi dataset menjadi data training dan

data testing.

test_size=0.2 berarti 20% data digunakan untuk testing dan 80% untuk training. dari total 200 baris, pembagiannya menjadi 160 data training dan 40 data testing. random_state=42 menjaga pembagian tetap sama ketika kode dijalankan kembali.

In [ ]:
```
# Membuat plot dari dataset
plt.scatter(X_train['TV'], y_train, label='TV Ad Budget') # Changed column name to
plt.scatter(X_train['Radio'], y_train, label='Radio Ad Budget') # Changed column na
plt.scatter(X_train['Newspaper'], y_train, label='Newspaper Ad Budget') # Changed c
plt.xlabel('Advertising Budget ($)')
plt.ylabel('Revenue ($)')
plt.legend()
plt.show()
```

setiap plt.scatter() membuat scatter plot antara satu fitur anggaran iklan dengan nilai sales. sumbu x menunjukkan anggaran iklan dan sumbu y menunjukkan penjualan. tiga kelompok titik memperlihatkan hubungan tv, radio, dan newspaper dengan sales.

pada gambar ini pemeriksaan awal untuk melihat pola, sebaran, dan kemungkinan hubungan antarvariabel sebelum model dibangun. pola titik tidak harus membentuk garis lurus karena mlp dapat mempelajari hubungan non-linear melalui hidden layer dan fungsi aktivasi.

hasilnya berupa kumpulan titik pada grafik. grafik ini belum merupakan hasil prediksi mlp, data yang ditampilkan berasal dari data training.

In [ ]:
```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

# Membangun model MLP
model = Sequential([
    Dense(512, input_shape=(3,), activation='relu'),  # Input layer
    Dense(512, activation='relu'),  # Hidden layer 1
    Dense(256, activation='relu'),  # Hidden layer 2
    Dense(1)  # Output layer (tidak perlu fungsi aktivasi karena ini masalah regres
])
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.summary()
```

cell ini membangun jaringan saraf multilayer perceptron (mlp) dengan sequential, yaitu susunan layer yang diproses secara berurutan. menurut modul, mlp memiliki input layer, hidden layer, dan output layer.

- layer pertama dense(512, input_shape=(3,), activation='relu') menerima tiga fitur: tv, radio, dan newspaper. 512 menunjukkan jumlah neuron pada layer tersebut. relu menerapkan fungsi max(0, x), yaitu nilai negatif menjadi nol dan nilai positif diteruskan.
- hidden layer berikutnya memiliki 512 neuron dan 256 neuron. layer-layer ini menggabungkan informasi dari fitur sebelumnya untuk mempelajari pola yang lebih

kompleks.

- output layer dense(1) menghasilkan satu angka prediksi sales. aktivasi linear bawaan digunakan karena masalahnya regresi, bukan klasifikasi kelas. pada proses forward propagation, nilai input diteruskan melewati layer, dikalikan dengan bobot, ditambah bias, lalu diproses oleh aktivasi sampai menghasilkan prediksi. model.compile() menentukan adam sebagai optimizer, mse sebagai loss, dan mae sebagai metrik. mse mengukur rata-rata kuadrat selisih prediksi dan nilai sebenarnya, sedangkan mae mengukur rata-rata selisih absolut.

ketika model.summary() dijalankan, output menampilkan susunan layer, bentuk output, dan jumlah parameter. parameter adalah bobot dan bias yang akan dipelajari.

In [ ]:
```
early_stop = tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=5)
history = model.fit(X_train, y_train, epochs=1400, validation_data=(X_test, y_test)
```

model.fit() menjalankan training menggunakan x_train dan y_train selama maksimal 1400 epoch. satu epoch berarti model melewati seluruh data training satu kali. validation_data meminta keras menghitung loss dan mae pada data validasi setelah setiap epoch yg sdah berjalan.

1. forward propagation: data input diteruskan melalui layer untuk menghasilkan prediksi.
2. menghitung loss: prediksi dibandingkan dengan target menggunakan mse.
3. backpropagation: kesalahan diteruskan mundur dari output menuju layer sebelumnya untuk menghitung gradien setiap bobot.
4. optimizer adam memperbarui bobot dan bias berdasarkan gradien agar loss semakin kecil.
5. proses diulang pada epoch berikutnya. output epoch menampilkan loss dan mae untuk training serta val_loss dan val_mae untuk validasi. nilai yang semakin kecil umumnya menunjukkan prediksi semakin dekat dengan target. early_stop sudah dibuat dengan memantau val_loss, tetapi callback tersebut belum dimasukkan ke argumen model.fit(). akibatnya, early stopping belum aktif dan training tetap mengikuti batas epoch yang ditentukan.

In [ ]:
```
test_loss, test_mae = model.evaluate(X_test, y_test)
print(f"Mean Absolute Error pada data pengujian: {test_mae}")
```

model.evaluate(x_test, y_test) menghitung performa model pada data testing. test_loss berisi nilai loss mse, sedangkan test_mae berisi rata-rata selisih absolut antara prediksi dan nilai sales sebenarnya.

output menunjukkan mae sekitar 1.2908. artinya, pada data uji ini, rata-rata selisih absolut prediksi sales terhadap nilai sebenarnya sekitar 1.29 satuan sales. semakin kecil mae, semakin dekat hasil prediksi dengan nilai sebenarnya. nilai ini perlu dibaca bersama grafik dan kondisi pembagian data, karena data testing juga dipakai sebagai validation saat training, hasil pengujian belum sepenuhnya independen.

In [ ]:
```
# Plot Loss dan MAE selama pelatihan
plt.figure(figsize=(12, 4))

# Plot Loss
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
```

history menyimpan catatan metrik dari setiap epoch. grafik pertama membandingkan training loss dan validation loss, sedangkan grafik kedua membandingkan training mae dan validation mae.

loss pada grafik ini menggunakan mse. jika kedua kurva menurun, model sedang memperbaiki prediksinya. jika training loss terus turun tetapi validation loss mulai naik, hal tersebut dapat menjadi tanda overfitting. jika keduanya masih tinggi atau tidak membaik, model mungkin belum mempelajari pola dengan baik.

In [ ]:
```
y_pred_test = model.predict(X_test)

plt.scatter(X_test['TV'], y_test, color='blue')

# Scatter plot of predictions (red)
plt.scatter(X_test['TV'], y_pred_test, color='red')

# Set the axis labels and title
plt.xlabel('TV Ad Budget ($)')
plt.ylabel('Sales ($)')
plt.title('Neural Network Regression - TV Ad Budget vs Sales')

# Add a legend to the plot
plt.legend(['True Values', 'Predictions'])
plt.show()
```

model.predict(x_test) meminta model menghasilkan prediksi untuk seluruh baris data testing. hasilnya disimpan pada y_pred_test. scatter plot kemudian membandingkan nilai sales sebenarnya dengan prediksi, dengan anggaran tv sebagai sumbu x.

titik biru menunjukkan nilai sales aktual, sedangkan titik merah menunjukkan hasil prediksi mlp. jika titik merah berdekatan dengan titik biru pada rentang anggaran yang sama, prediksi model relatif mendekati nilai aktual. jarak yang jauh menunjukkan salah dalam memprresdiksi yang lebih besar.

grafik ini hanya memilih tv sebagai sumbu x untuk membantu membaca hubungan secara visual. prediksi model sebenarnya menggunakan ketiga fitur sekaligus, yaitu tv, radio, dan newspaper.

In [ ]:
```
plt.scatter(X_test['Radio'], y_test, color='blue')

# Scatter plot of predictions (red)
plt.scatter(X_test['Radio'], y_pred_test, color='red')

# Set the axis labels and title
plt.xlabel('Radio Ad Budget ($)')
plt.ylabel('Sales ($)')
plt.title('Neural Network Regression - Radio Ad Budget vs Sales')

# Add a legend to the plot
plt.legend(['True Values', 'Predictions'])
plt.show()
```

cell ini menggunakan y_pred_test yang sama, tetapi menampilkan anggaran radio pada sumbu x. titik biru adalah sales aktual dan titik merah adalah sales hasil prediksi.

tujuannya membandingkan kedekatan prediksi dan nilai aktual ketika data dilihat berdasarkan anggaran radio. perlu diingat bahwa model tidak memprediksi sales hanya dari radio; model menerima tiga fitur iklan secara bersamaan. karena itu, grafik ini merupakan cara visualisasi, bukan model regresi satu fitur.

In [ ]:
```
plt.scatter(X_test['Newspaper'], y_test, color='blue')

# Scatter plot of predictions (red)
plt.scatter(X_test['Newspaper'], y_pred_test, color='red')

# Set the axis labels and title
plt.xlabel('Newspaper Ad Budget ($)')
plt.ylabel('Sales ($)')
plt.title('Neural Network Regression - Newspaper Ad Budget vs Sales')

# Add a legend to the plot
plt.legend(['True Values', 'Predictions'])
plt.show()
```

cell ini membuat perbandingan yang sama dengan dua grafik sebelumnya, tetapi menggunakan anggaran newspaper sebagai sumbu x. titik biru menunjukkan nilai sales sebenarnya dan titik merah menunjukkan hasil prediksi.

grafik membantu melihat apakah prediksi model berada dekat dengan nilai aktual pada berbagai tingkat anggaran newspaper. kesimpulan performa tetap perlu mempertimbangkan ketiga fitur dan metrik mae, karena model mlp menggunakan tv, radio, dan newspaper secara bersama-sama.